

You are a senior QA engineer and TDD architect. We are building CareOrbit using:

| Layer                 | Choice                                      |
|-----------------------|---------------------------------------------|
| Backend               | Python 3.12 + FastAPI 0.111                 |
| Frontend              | React 18 + Vite 5                           |
| Monorepo              | Turborepo (web + future mobile)             |
| Styling               | Tailwind CSS 3.4 + NativeWind ready         |
| State                 | Zustand + TanStack Query v5                 |
| Database              | Azure PostgreSQL Flexible Server + pgcrypto |
| All 10 Azure services | As decided earlier                          |

Here is every service, what it does specifically in CareOrbit, why it was chosen over alternatives, the exact package, and the mock path for tests.

| # | Service                        | What It Does in CareOrbit                    | Why This Over Alternatives                       | Package                               | Test Mock Path                                        |
|---|--------------------------------|----------------------------------------------|--------------------------------------------------|---------------------------------------|-------------------------------------------------------|
| 1 | Azure AI Document Intelligence | OCR on prescription photos, medicine strips, | Chosen over Computer Vision Read API because the | azure-ai-document-intelligence==1.0.0 | services.azure_vision.AzureVisionService.extract_text |

lab  
reports  
—  
extracts  
structur  
ed  
tables,  
handwri  
ting,  
printed  
text  
prebuilt  
-documen  
t model  
understand  
s medical  
document  
layout (lab  
report  
tables,  
key-value  
pairs like  
"HbA1c:  
7.8%")  
natively

2 Azure  
Open  
AI  
(GPT-  
4o)

Structur  
ed  
medicatio  
ion  
extracti  
on from  
OCR  
text,  
care  
gap  
reasoni  
ng,  
multi-ag  
ent  
respons  
e  
synthes  
is, Hindi  
medical  
GPT-4o  
over  
GPT-3.5:  
Hindi  
abbreviatio  
ns  
(BD/OD/H  
S/SOS),  
brand-to-g  
eneric  
mapping  
(Thyronor  
m→Levoth  
yroxine),  
multi-hop  
reasoning  
for PHIG

openai=  
1.35.0

```
services.azure_opena  
i.AzureOpenAIService  
.extract_structured_  
data
```

term  
understand

3 Azure AI Translator

Hindi ↔ English for patient queries and AI responses

No alternative — only service with Indian language support at this quality level. Uses `httpx` directly (no `async` Azure Translator SDK exists)

`httpx==0.27.0`

`services.azure_translator.httpx.AsyncClient`

4 Azure AI Language (Healthcare NER)

Extract medication names, dosages, diagnoses from OCR text with direct RxNorm and

Chosen over generic NER because it returns `dataSourceEntityId` pointing directly to RxNorm codes — eliminates

`azure-ai-textanalytics==5.3.0`

`services.azure_language.TextAnalyticsClient`

ICD-10 the  
entity fuzzy-match  
linking h step for  
well-structu  
red text

5 Azure  
e  
Blob  
Storage

Store  
upload  
d  
prescription  
images;  
serve  
generated  
health  
summary  
PDFs  
via  
time-lim  
ited  
SAS  
URLs

Required  
— no  
alternative.  
PDFs can't  
live in  
Postgres  
(size +  
streaming)

```
azure-storage-blob==12.20.0
```

```
services.azure_blob.BlobServiceClient
```

6 Azure  
e  
Communication  
Services  
(Email)

Medication  
reminder  
emails,  
upload  
result  
notifications,  
drug

Chosen  
over  
SendGrid:  
single  
Azure  
billing,  
HIPAA-eligible service  
tier,  
begin\_se

```
azure-communication-email==1.0.0
```

```
services.azure_email.EmailClient.begin_send_email
```

interacti  
on alert  
emails  
nd()  
returns a  
poller  
(async-frie  
ndly)

7 **Azur  
e AI  
Search  
h**

RAG  
for drug  
interacti  
on  
lookup  
(10K+  
drug  
pairs  
indexed  
with  
vectors)  
and  
clinical  
guidelin  
e  
retrieval  
for care  
gap  
detectio  
n  
Chosen  
over  
Pinecone/  
Weaviate:  
hybrid  
keyword+v  
ector  
search  
catches  
both  
spelling  
variants of  
drug  
names  
AND  
semantic  
similarity.  
Stays in  
Azure  
billing

azure-se  
arch-doc  
uments=  
11.6.0

services.azure\_search  
h.SearchClient.search

8 **Azur  
e  
Post  
greS  
QL  
Flexi  
ble**

All  
relation  
al data  
—  
users,  
PHIG  
nodes/e  
dges,  
Chosen  
over  
Cosmos  
DB:  
pgcrypto  
extension  
for

asyncpg=  
=0.29.0  
+  
sqlalchemy  
my==2.0.  
30

Real DB in Docker for  
integration tests  
(pytest-asyncio)

Server  
audit\_log,  
reminders,  
subscriptions,  
usage tracking  
pgp\_sym\_  
encrypt  
PII  
encryption  
at rest,  
ACID  
transaction  
s for  
append-only  
audit\_log,  
HIPAA  
BAA  
available

9 Azure Key Vault  
ENCRYPT  
PTION  
\_KEY  
(for  
pgcrypto),  
JWT\_SECRET,  
all  
Azure  
API  
keys  
Required  
for  
production:  
secret  
rotation  
without  
redeployment,  
audit  
trail for  
every key  
access,  
HIPAA  
requirements  
azure-keyvault-secrets==4.8.0  
services.azure\_keyvault.SecretClient.get\_secret

10 Azure Static Web Apps  
Hosting  
Next.js  
frontend  
d  
First-class  
Next.js  
support,  
built-in  
staging  
slots,  
(deployment target, no SDK)

global  
CDN, free  
tier  
appropriate  
for  
demo/com  
petition

DO NOT WRITE ANY APPLICATION CODE YET.

Your only job in this prompt is to:

1. Produce an IMPLEMENTATION PLAN
2. Generate all TEST CASES as code for human review BEFORE implementation

\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_  
PHASE A — IMPLEMENTATION PLAN (ordered by TDD discipline)  
\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_

Produce a SPRINT-BY-SPRINT plan (6 sprints × 1 week each) where:

- Sprint N tests are written BEFORE Sprint N code
- Each sprint has: Goal, Test files to write first, Code files to write second, Azure services introduced, Definition of Done (all tests GREEN)
- No sprint skips a test-first step

Sprint structure:

Sprint 1: Infrastructure + Auth (Postgres, JWT, rate limiting, audit trail)  
Sprint 2: Document Pipeline (Azure Vision + Language + OCR → PHIG nodes)  
Sprint 3: Drug Interaction + Care Gap Engine (Azure Search RAG + PHIG graph)

- Sprint 4: AI Chat + Multi-Agent Orchestrator (Azure OpenAI, Hindi support)
- Sprint 5: Summaries + Reminders + Subscriptions (Azure Blob, Email, tier gates)
- Sprint 6: Frontend + E2E (Next.js dashboard, Playwright journeys, Ramesh E2E)

For each sprint, output this exact table:

| Step     | File                           | Type           | Azure Service Mocked | Runs Green When                  |
|----------|--------------------------------|----------------|----------------------|----------------------------------|
| 1 (TEST) | tests/unit/test_auth_tokens.py | Unit           | None (pure JWT)      | JWT encode/decode contract holds |
| 2 (CODE) | api/middleware/auth.py         | Implementation | None                 | Step 1 tests pass                |
| ... etc  |                                |                |                      |                                  |

PHASE B — GENERATE ALL TEST FILES (for human review)

Generate the following test files in full, production-quality Python.  
Every file must follow these absolute rules:

RULE 1 — NO CONDITIONAL GUARDS

- ✗ NEVER: if response.status\_code == 200: assert ...
- ✗ NEVER: if mock.called: assert ...
- ✓ ALWAYS: assert response.status\_code == 200
- ✓ ALWAYS: assert mock.call\_count >= 1

RULE 2 — NO TAUTOLOGICAL MOCK TESTS

- ✗ NEVER: patch the system under test, then assert the mock's return value



✓ ALWAYS: patch only the external dependency (Azure SDK), run real logic

### RULE 3 — NO STUB TESTS

✗ NEVER: `def test_something(): pass` (or body with only comments)

✓ ALWAYS: if feature not built yet, use `@pytest.mark.xfail(strict=True, reason="...")`

### RULE 4 — AZURE MOCKS MUST USE EXACT SDK PATHS

Every `@patch` or `AsyncMock` must target the actual import path used in the route:

✓ `patch("api.routes.documents.document_intelligence_client.begin_analyze_document")`

✗ `patch("azure.ai.documentintelligence.DocumentIntelligenceClient")`

### RULE 5 — PHASE MARKERS

`@pytest.mark.phase2` for OTP, voice input, FHIR export

`@pytest.mark.phase3` for Stripe/Razorpay payment processing

`@pytest.mark.demo_only` for upgrade flow (no payment)

`@pytest.mark.slow` for real Postgres integration tests

### RULE 6 — UNIQUE STATE PER TEST

Every test that registers a user must use a uuid4-suffixed email:

`email = f"test.{uuid4().hex[:8]}@careorbit.dev"`

### FILES TO GENERATE

Generate each file as a fenced code block with the filepath as the label.

#### FILE 1: tests/conftest.py

Include:

- All Azure service fixtures (`mock_vision`, `mock_openai`, `mock_translator`,

mock\_language, mock\_blob, mock\_email, mock\_search)

- Each fixture must use AsyncMock with the EXACT method signatures from the Azure Python SDK (not generic mocks)
- Azure Vision fixture: mock begin\_analyze\_document() returning a poller with .result() → AnalyzeResult with pages[].lines[].content
- Azure OpenAI fixture: mock client.chat.completions.create() returning ChatCompletion with choices[0].message.content as JSON string
- Azure Search fixture: mock SearchClient.search() returning an iterator of SearchResult objects with @search.score and drug interaction fields
- PHASE1\_SOURCES constant
- patient\_ramesh and patient\_priya fixtures with all fields

FILE 2: tests/unit/test\_confidence\_scoring.py

Cover: SOURCE\_CEILINGS for all Phase 1 sources, ceiling enforcement, determinism (10 calls same result), ranking order, never-negative, patient\_confirmed as Phase 1 source (hardcoded 0.85 in confirmations.py)

FILE 3: tests/unit/test\_drug\_database.py

Cover: All 10 RxNorm codes (Metformin=6809, Amlodipine=17767, Atorvastatin=83367, Aspirin=1191, Ibuprofen=5640, Telmisartan=73494, Levothyroxine=10582, Losartan=202272 INGREDIENT-LEVEL, Omeprazole=7646, Paracetamol=161), fuzzy matching, Indian brand → generic

FILE 4: tests/unit/test\_encryption.py

Cover: encrypt\_sql() is pass-through (not pgp\_sym\_encrypt wrapper), get\_encryption\_params() injects key, registration INSERT uses pgp\_sym\_encrypt — assert call\_count >= 2 (SELECT duplicate check + INSERT), never if .called

FILE 5: tests/unit/test\_auth\_tokens.py

Cover: JWT sub field, token type, 15-min expiry, bcrypt format (\$2b\$),

verify correct/wrong password, different users → different tokens,  
tampered token rejected

FILE 6: tests/functional/test\_api\_auth.py

Cover: register → 200 + tokens immediately (NO OTP in Phase 1),  
duplicate email → 409, weak password → 422, login success,  
login wrong password → 401, refresh via JSON body (not cookie),  
logout via JSON body

FILE 7: tests/functional/test\_api\_documents.py +  
test\_api\_documents\_boundaries.py

Cover: upload prescription → 200 + nodes\_created ≥ 2,  
interaction detected unconditionally (no 'or care\_gaps ≥ 0'),  
needs\_confirmation for low confidence,  
file > 10MB → 400, PDF MIME type → 400, missing file field → 422

FILE 8: tests/functional/test\_api\_reminders.py

Cover: POST /api/reminders/create with reminder\_time +  
days\_of\_week=[int],  
GET /api/reminders/list returns list,  
DELETE uses REAL reminder\_id from create (hard assert create  
succeeded first),  
nonexistent medication\_node\_id → 404,  
caregiver with 'view' permission → 403 on create

FILE 9: tests/security/test\_refresh\_token\_security.py

Cover: SHA-256 hash stored (assert call\_count ≥ 2, inspect INSERT  
params),  
revoked token rejected, rotation invalidates old token,  
forged token → 401/403

FILE 10: tests/security/test\_audit\_trail.py

Cover: user\_id/patient\_id/IP/action recorded via direct param access  
(call\_args[0][1], NOT str(call\_args)),  
REGISTER action present in SCHEMA\_VALID\_ACTIONS,

audit INSERT never executes UPDATE or DELETE

FILE 11: tests/security/test\_cross\_patient\_rbac.py

Cover: Patient A token → Patient B medications → 403,

Patient A → Patient B overview → 403,

Patient A → own data → 200 (must succeed),

5 endpoints tested with cross-patient access

FILE 12: tests/security/test\_rate\_limiting.py

Cover: Falsifiable design — register real user, 5 wrong attempts,

correct password on 6th → 429 (not 200),

different IP not rate limited

FILE 13: tests/functional/test\_api\_summary.py

Cover: GET (not POST) → StreamingResponse,

content-type=application/pdf,

magic bytes b"%PDF", content-disposition header,

empty patient (total\_nodes=0) → HTTP 200 + JSON {"error": ...} (not PDF, not 404),

caregiver 'view' → can generate summary

FILE 14: tests/functional/test\_api\_caregivers.py

Cover: add →

{"status": "added", "caregiver\_name": ..., "permission\_level": ...}

(NO link\_id in response),

delete is idempotent → 200 always,

'neighbor' relationship → 400, 'admin' permission → 400,

unregistered email → 404,

caregiver limit @pytest.mark.xfail(strict=True) with real gate mock

FILE 15: tests/functional/test\_api\_subscriptions.py

Cover: /current and /plans use standard client (NO include\_router hack — subscriptions router IS in main.py v0.3.0),

free tier shows\_ads=True,

plans endpoint returns 3 tiers with correct prices (0, 799, 1999 cents),

upgrade journey @pytest.mark.demo\_only only

FILE 16: tests/functional/test\_api\_health.py

Cover: GET /health → 200, status="healthy", service="careorbit-api",  
version field present, no auth required

FILE 17: tests/integration/test\_document\_state\_machine.py

Cover: process\_document() always returns terminal state (success/  
needs\_confirmation/failed — NEVER "processing"),  
vision error → failed + error\_message set,  
low ocr confidence → needs\_confirmation,  
high confidence → success

FILE 18: tests/integration/test\_orchestrator.py

Cover: medication keywords → MedicationAgent in agents\_used (strict,  
NO 'or len >= 1' fallback),  
care gap keywords → CareGapAgent,  
no keyword match → all 3 agents,  
Hindi → translator.translate called,  
response has message/agents\_used/alerts/confidence fields

FILE 19: tests/false\_positive\_negative/test\_interaction\_false\_positives.py

+ tests/false\_positive\_negative/test\_interaction\_false\_negatives.py

SPLIT into Wiring tests (mock pipeline → test API contract) and  
Algorithm tests (mock ONLY Azure Search → run real phig\_builder  
detection):

FP Algorithm: search returns [] → engine produces 0 alerts

FN Algorithm: search returns interaction data → engine MUST produce  
alert

FN Algorithm: renal impairment severity modifier → escalates severity

Label each class clearly: "WIRING TEST" vs "ALGORITHM TEST  
(CLINICAL SAFETY)"

FILE 20: tests/regression/test\_lab\_trend\_edge\_cases.py

Cover: All 4 tests as @pytest.mark.xfail(strict=False) with stub logic

that imports the module and catches ImportError (not bare assert False)

FILE 21: tests/business\_logic/test\_tier\_config.py

Cover: All 3 tiers defined, shows\_ads flags, document/summary/caregiver limits,

data retention months, family\_dashboard flag, get\_tier\_limits unknown → defaults free,

check\_feature\_allowed, feature gate check\_usage at/below/above limit

FILE 22: tests/e2e/test\_patient\_journey\_ramesh.py

Cover: All 8 steps (register → upload prescription → upload ibuprofen → upload lab → confirm → care gaps → generate PDF → chat EN → chat HI),

interaction assertion unconditional (no 'or care\_gaps >= 0'),

summary uses GET not POST, PDF magic bytes check

FILE 23: tests/coverage\_matrix.py

Cover: 17 P0 features (14 original + History Agent/Lab Trends/Adherence promoted in v0.3.0), 1 P1 feature (Family Caregiver Dashboard),

safety features have false\_positive AND false\_negative test types,

auth has >= 5 security test files

\_\_\_\_\_

PHASE C — HUMAN REVIEW CHECKLIST (output for developer)

\_\_\_\_\_

After generating all test files, output a REVIEW CHECKLIST table:

For each test file, list:

| File | Test Count | Azure Service Mocked | SDK Mock Path | Conditional Guards? | Tautological? |
|------|------------|----------------------|---------------|---------------------|---------------|
|------|------------|----------------------|---------------|---------------------|---------------|

Then output a summary:

- Total test count
- Services with mocks verified against real SDK method signatures
- Phase markers applied correctly?
- Any tests that need business/clinical domain review before implementation?

End with: "READY FOR HUMAN REVIEW — Do not proceed to Prompt 3 until approved."

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

### **PROMPT 3 OF 3 — FULL TDD BUILD WITH AZURE SERVICE INTEGRATION**

**(Paste ONLY after human has reviewed and approved all test cases)**

\_\_\_\_\_

\_\_\_\_\_

You are a senior full-stack engineer. Human has reviewed and approved all test cases.

Now implement CareOrbit following strict TDD: for each Sprint, write failing tests first,

then write the minimum code to make them pass, then refactor.

**APPROVED TEST SUITE:** All 23 test files from Prompt 2 are in /tests/  
**STACK CONFIRMED:**

Backend: Python 3.12 + FastAPI 0.111 + SQLAlchemy 2.0 async +  
asyncpg

Frontend: Next.js 14.2 App Router + TypeScript 5.4 + Tailwind CSS 3.4

DB: Azure PostgreSQL Flexible Server + pgcrypto extension

Cache: Redis (Azure Cache for Redis) — rate limiting + session

Queue: Azure Service Bus — async reminder email delivery

## AZURE SERVICE INTEGRATION INSTRUCTIONS (apply to every service)

### 1. AZURE AI VISION (Document Intelligence)

Package: azure-ai-documentintelligence==1.0.0

Why this over Computer Vision: prebuilt-document model understands medical document layout (tables, key-value pairs, handwriting).

Async polling is critical — OCR takes 3-8 seconds.

Implementation pattern:

```
```python
# services/azure_vision.py
from azure.ai.documentintelligence.aio import
DocumentIntelligenceClient
from azure.core.credentials import AzureKeyCredential

class AzureVisionService:
    def __init__(self):
        self.client = DocumentIntelligenceClient(
            endpoint=settings.AZURE_VISION_ENDPOINT,
            credential=AzureKeyCredential(settings.AZURE_VISION_KEY)
        )

    async def extract_text(self, image_bytes: bytes) -> OCRResult:
        async with self.client:
            poller = await self.client.begin_analyze_document(
                "prebuilt-read",
                analyze_request=image_bytes,
```



```

        content_type="image/jpeg"
    )
    result = await poller.result()
    full_text = "\n".join(
        line.content
        for page in result.pages
        for line in (page.lines or [])
    )
    avg_confidence = sum(
        word.confidence
        for page in result.pages
        for line in (page.lines or [])
        for word in (line.words or [])
        if word.confidence
    ) / max(1, sum(
        len(line.words or [])
        for page in result.pages
        for line in (page.lines or [])
    ))
    return OCRResult(
        full_text=full_text,
        lines=[l.content for p in result.pages for l in (p.lines or [])],
        avg_confidence=avg_confidence,
        page_count=len(result.pages)
    )
'''

```

```

Test mock pattern:
'''python
# In conftest.py
@pytest.fixture
def mock_vision():
    service = AsyncMock(spec=AzureVisionService)
    mock_result = MagicMock()
    mock_result.pages = [MagicMock()]

```

```

        mock_result.pages[0].lines = [
            MagicMock(content="Tab Metformin 500mg BD", words=[
                MagicMock(content="Tab", confidence=0.97),
                MagicMock(content="Metformin", confidence=0.95),
            ])
        ]

        service.extract_text = AsyncMock(return_value=OCRResult(
            full_text="Tab Metformin 500mg BD",
            lines=["Tab Metformin 500mg BD"],
            avg_confidence=0.94,
            page_count=1
        ))

        service.classify_document_type =
        AsyncMock(return_value="prescription")

        return service

'''

```

CI/CD note: In GitHub Actions, use Docker with azurite (Azure emulator) for Blob Storage. Vision and OpenAI require real Azure credentials in integration tests — use GitHub Secrets, never committed .env.

## 2. AZURE OPENAI SERVICE (GPT-4o)

Package: openai==1.35.0 (Azure OpenAI uses same package since v1.0)  
 Why GPT-4o over GPT-3.5: medical entity extraction requires context understanding for Hindi/English drug names, dosage abbreviations (BD=twice daily).

Implementation pattern:

```

'''python
# services/azure_openai.py
from openai import AsyncAzureOpenAI

class AzureOpenAIService:
    def __init__(self):
        self.client = AsyncAzureOpenAI(

```

```

        azure_endpoint=settings.AZURE_OPENAI_ENDPOINT,
        api_key=settings.AZURE_OPENAI_API_KEY,
        api_version="2024-02-01"
    )

    self.deployment = settings.AZURE_OPENAI_DEPLOYMENT #
    "gpt-4o"

```

```

    async def extract_structured_data(
        self, ocr_text: str, doc_type: str
    ) -> dict:
        response = await self.client.chat.completions.create(
            model=self.deployment,
            messages=[
                {"role": "system", "content":
EXTRACTION_PROMPT[doc_type]},
                {"role": "user", "content": ocr_text}
            ],
            response_format={"type": "json_object"},
            temperature=0.1, # Low temperature for medical extraction
            max_tokens=2000
        )
        return json.loads(response.choices[0].message.content)
'''

```

```

Test mock pattern:
```python
@pytest.fixture
def mock_openai():
    service = AsyncMock(spec=AzureOpenAIService)
    service.extract_structured_data = AsyncMock(return_value={
        "medications": [
            {"name": "Metformin", "dosage": "500mg", "frequency": "BD",
            "drug_match_score": 0.98, "needs_confirmation": False}
        ],
        "doctor_name": "Dr. Amit Roy",

```

```

        "diagnoses": ["Type 2 Diabetes Mellitus"],
        "date": "15/01/2026"
    })
    service.chat = AsyncMock(return_value="Mocked AI response")
    return service
'''

```

Critical for testing: Always mock at the service class level, not the `openai.AsyncAzureOpenAI` level. This ensures tests verify your service's response contract, not OpenAI's SDK internals.

### 3. AZURE AI TRANSLATOR

Package: Use `requests/httpx` directly (no official async Azure Translator SDK)

Why `httpx` over `requests`: native async support, same API surface.

Implementation pattern:

```

'''python
# services/azure_translator.py
import httpx

class AzureTranslatorService:
    BASE_URL = "https://api.cognitive.microsofttranslator.com"

    async def translate(self, text: str, target: str, source: str = None) -> str:
        params = {"api-version": "3.0", "to": target}
        if source:
            params["from"] = source
        async with httpx.AsyncClient() as client:
            response = await client.post(
                f"{self.BASE_URL}/translate",
                params=params,
                headers={
                    "Ocp-Apim-Subscription-Key":
settings.AZURE_TRANSLATOR_KEY,

```



```

        rxnorm_id=next(
            (l.id for l in entity.data_sources
             if l.name == "RxNorm"), None
        )
    ))
    return entities
'''

```

## 5. AZURE BLOB STORAGE

Package: azure-storage-blob==12.20.0

Use async client for all operations.

Implementation pattern:

```

```python
from azure.storage.blob.aio import BlobServiceClient

    async def upload_document(self, image_bytes: bytes, patient_id: str,
doc_id: str) -> str:
        async with BlobServiceClient.from_connection_string(
            settings.AZURE_STORAGE_CONNECTION_STRING
        ) as client:
            container = client.get_container_client("prescriptions")
            blob_name = f"{patient_id}/{doc_id}.jpg"
            await container.upload_blob(blob_name, image_bytes,
overwrite=True)

            # Generate time-limited SAS URL (1 hour)
            sas_token = generate_blob_sas(
                account_name=client.account_name,
                container_name="prescriptions",
                blob_name=blob_name,
                account_key=settings.AZURE_STORAGE_KEY,
                permission=BlobSasPermissions(read=True),
                expiry=datetime.utcnow() + timedelta(hours=1)
            )

```

```

        return
    f"https://{client.account_name}.blob.core.windows.net/prescriptions/{blob_name}?{sas_token}"
    """

```

## 6. AZURE COMMUNICATION SERVICES (Email)

Package: azure-communication-email==1.0.0

Implementation pattern:

```

``python
from azure.communication.email import EmailClient

async def send_medication_reminder(
    self, to_email: str, patient_name: str,
    medication_name: str, dosage: str, time_label: str
) -> bool:
    client = EmailClient.from_connection_string(
        settings.AZURE_COMM_CONNECTION_STRING
    )
    message = {
        "senderAddress": settings.AZURE_COMM_SENDER_EMAIL,
        "recipients": {"to": [{"address": to_email}]},
        "content": {
            "subject": f"CareOrbit Reminder: {medication_name} —
{time_label}",
            "plainText": REMINDER_TEMPLATE_TEXT.format(
                name=patient_name, med=medication_name, dose=dosage
            ),
            "html": REMINDER_TEMPLATE_HTML.format(
                name=patient_name, med=medication_name, dose=dosage
            )
        }
    }
    poller = client.begin_send(message)
    result = poller.result()

```

```
return result.status == "Succeeded"
```

```
'''
```

## 7. AZURE AI SEARCH (RAG for Drug Interactions + Guidelines)

Package: azure-search-documents==11.6.0

Index design:

- drug\_interactions index: drug\_a, drug\_b, severity, description, clinical\_action, severity\_modifiers (JSON), embedding (text-embedding-3-small)
- clinical\_guidelines index: condition, screening\_type, frequency, evidence\_grade, source, applicable\_codes (ICD-10 array), embedding

Implementation pattern:

```
```python
```

```
from azure.search.documents.aio import SearchClient
```

```
from azure.search.documents.models import VectorizedQuery
```

```
async def search_drug_interactions(
    self, drug_a: str, drug_b: str
) -> list[DrugInteraction]:
    async with SearchClient(
        self.endpoint, "drug-interactions",
        AzureKeyCredential(self.search_key)
    ) as client:
        # Hybrid search: keyword + vector
        embedding = await self.get_embedding(f"{drug_a} {drug_b}
interaction")
        vector_query = VectorizedQuery(
            vector=embedding, k_nearest_neighbors=3,
            fields="embedding"
        )
        results = await client.search(
            search_text=f"{drug_a} {drug_b}",
            vector_queries=[vector_query],
```



```

        filter=f"drug_a eq '{drug_a}' or drug_b eq '{drug_a}'",
        top=5
    )
    return [
        DrugInteraction(**result)
        async for result in results
        if result["@search.score"] > 0.7
    ]
'''

```

## SPRINT 1 BUILD — INFRASTRUCTURE + AUTH (TDD)

Step 1 (FAIL): Run tests/unit/test\_auth\_tokens.py → all RED

Step 2 (CODE): Implement api/middleware/auth.py (JWT, bcrypt, create\_refresh\_token)

Step 3 (PASS): Run test\_auth\_tokens.py → all GREEN

Step 4 (FAIL): Run tests/functional/test\_api\_auth.py → all RED

Step 5 (CODE): Implement api/routes/auth.py (register, login, refresh, logout)

- register: check duplicate email, hash password, INSERT with pgp\_sym\_encrypt,

- create refresh token, return access\_token + refresh\_token immediately

(NO OTP in Phase 1 — Phase 2 gate exists in test\_api\_otp.py @phase2)

- login: verify password with bcrypt, check is\_active, return tokens

- refresh: verify token SHA-256 hash, check not revoked, rotate token

- logout: revoke specific token by hash

Step 6 (PASS): Run test\_api\_auth.py → all GREEN

POSTGRES SCHEMA — create in database/schema.sql:

```

```sql

```

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- REQUIRED for
pgp_sym_encrypt
```

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,
  -- PII ENCRYPTED at rest:
  email TEXT NOT NULL, -- stored as pgp_sym_encrypt(email,
:encryption_key)
  phone_number TEXT, -- stored as pgp_sym_encrypt(phone,
:encryption_key)
  password_hash TEXT NOT NULL,
  date_of_birth DATE,
  gender VARCHAR(10),
  preferred_language VARCHAR(5) DEFAULT 'en',
  subscription_tier subscription_tier DEFAULT 'free',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE refresh_tokens (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  token_hash VARCHAR(64) NOT NULL UNIQUE, -- SHA-256 of raw
token
  ip_address INET,
  user_agent TEXT,
  expires_at TIMESTAMPTZ NOT NULL,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- audit_log: APPEND ONLY — no UPDATE/DELETE permissions for app
user
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL,
  patient_id UUID NOT NULL,
  action VARCHAR(50) NOT NULL,
  resource_type VARCHAR(50),
  resource_id UUID,
  ip_address INET,
  user_agent TEXT,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
  -- NO updated_at — this table is append-only by design
);
'''
```

## SPRINT 2 BUILD — DOCUMENT PIPELINE (TDD)

Step 1 (FAIL): Run `tests/integration/test_document_state_machine.py` → all RED

Step 2 (CODE): Implement `pipeline/document_pipeline.py` with these rules:

- `process_document()` ALWAYS returns terminal state  
(success | needs\_confirmation | failed)
- "processing" is an internal state NEVER returned to callers
- Vision timeout → failed with error\_message
- Any OCR confidence < 0.30 → document\_type="unreadable",  
status="failed"
- Med confidence < threshold AND not critical → needs\_confirmation item

Step 3 (PASS): Run `test_document_state_machine.py` → all GREEN

Step 4 (FAIL): Run `tests/functional/test_api_documents.py` → all RED

Step 5 (CODE): Implement `api/routes/documents.py` with:

- Allowed MIME types: `image/jpeg`, `image/png`, `image/webp`, `image/heic`
- Size limit: 10MB (check `file.size` before reading bytes)
- Feature gate: `await enforce_document_limit(current_user["id"])`
- Processing priority: 2-second yield for free tier

Step 6 (PASS): Run all document tests → GREEN

## SPRINT 3 BUILD — INTERACTION ENGINE (TDD)

Step 1 (FAIL): Run `tests/false_positive_negative/` → all RED

NOTE: These are split into WIRING (mock pipeline) and ALGORITHM (mock only Azure Search)

The algorithm tests run the real `phig_builder` detection logic.

They will fail because `phig_builder` doesn't exist yet.

Step 2 (CODE): Implement `graph/phig_builder.py`:

- `check_interactions_for_node()`: calls Azure Search, applies `severity_modifiers`
- Severity modifiers: `renal_impairment` escalates Moderate → HIGH if `creatinine > 1.3`
- `evaluate_care_gaps()`: RAG lookup against `clinical_guidelines_index`

Step 3 (PASS): Wiring tests pass (mocked pipeline). Algorithm tests pass (real engine, mocked Search).

## SPRINT 4 BUILD — AI CHAT + ORCHESTRATOR (TDD)

Step 1 (FAIL): Run `tests/integration/test_orchestrator.py` → all RED

Step 2 (CODE): Implement `agents/orchestrator.py`:

- Keyword routing: MEDICATION\_KEYWORDS → MedicationAgent,  
CARE\_GAP\_KEYWORDS → CareGapAgent, HISTORY\_KEYWORDS  
→ HistoryAgent  
- No keyword match → ALL 3 agents (never empty agents\_used)  
- Hindi detection: language != "en" → translate before agent processing  
Step 3 (PASS): Run test\_orchestrator.py → all GREEN

## SPRINT 5 BUILD — SUMMARIES + REMINDERS (TDD)

Step 1 (FAIL): Run tests/functional/test\_api\_summary.py → all RED

Step 2 (CODE): Implement api/routes/summary.py:

- GET /api/summary/generate (NOT POST)
- Returns StreamingResponse(PDF) when total\_nodes > 0
- Returns HTTP 200 + JSON {"error": "..."} when total\_nodes == 0  
(NOT HTTP 404 — use 'return dict' not 'raise HTTPException')

Step 3 (PASS): Run test\_api\_summary.py → GREEN

Step 4 (FAIL): Run tests/functional/test\_api\_reminders.py → all RED

Step 5 (CODE): Implement api/routes/reminders.py:

- POST /api/reminders/create: reminder\_time (str "HH:MM"),  
days\_of\_week (list[int] 1-7), RBAC requires "edit"
- GET /api/reminders/list: returns list of reminder dicts
- DELETE /api/reminders/{reminder\_id}: soft-delete (is\_active=False)  
returns 404 if id not found

Step 6 (PASS): Run test\_api\_reminders.py → GREEN

CRITICAL: test\_delete\_existing\_reminder asserts create.status\_code ==  
200

first. If create fails, DELETE test must also fail (no silent skip).

[illegible]

- Next.js 14 App Router (not Pages Router) — server components for data fetching

careorbit-web/src/ — app/ | — (auth)/ | — login/page.tsx | |  
— register/page.tsx | — (dashboard)/ | — layout.tsx # Sidebar +  
nav with tier badge | | — page.tsx # Health overview (PHIG summary)  
| | — medications/page.tsx | | — labs/page.tsx | | —  
reminders/page.tsx | | — caregivers/page.tsx | | — chat/page.tsx |  
| — settings/ | | — profile/page.tsx | | — subscription/page.tsx |  
— api/ | — [...proxy]/route.ts # Optional: API proxy for CORS —  
components/ | — ui/ # shadcn components | — medication-card.tsx  
| — interaction-alert.tsx # Red alert for drug interactions | —  
confidence-badge.tsx # Color-coded confidence score | —  
upload-zone.tsx # Drag-and-drop document upload | — tier-gate.tsx #  
Upgrade prompt for gated features | — ad-slot.tsx # Ad container (free  
tier only) | — lib/ | — api.ts # Typed API client + auto token refresh |  
— auth.ts # JWT decode, session management | — types.ts #  
Generated from OpenAPI spec — tests/ | — components/ # Vitest +  
RTL component tests — e2e/ # Playwright journey tests

```
``typescript
// tests/e2e/ramesh-journey.spec.ts
import { test, expect } from "@playwright/test";

test.describe("Ramesh Kumar — Complete Patient Journey", () => {
  test("Step 1: Register as free tier user", async ({ page }) => {
    await page.goto("/register");
    await page.fill('[name="name"]', "Ramesh Kumar");
    await page.fill('[name="email"]', `ramesh.e2e.${Date.now()}@test.com`);
    await page.fill('[name="password"]', "RameshPass123!");
    await page.fill('[name="phone_number"]', "+919876543210");
    await page.click('[type="submit"]');
    await expect(page).toHaveURL("/dashboard");
    // Free tier: ads must be visible
    await expect(page.locator('[data-testid="ad-slot"]')).toBeVisible();
  });
});
```

```
test("Step 2: Upload prescription → see nodes created", async ({ page })
=> {
  // Log in as Ramesh first
  await page.goto("/dashboard/medications");
  const uploadZone = page.locator('[data-testid="upload-zone"]');
  await uploadZone.setInputFiles("tests/fixtures/prescription.jpg");
  await expect(page.locator('[data-testid="processing-status"]'))
    .toContainText("success", { timeout: 15000 });
  await expect(page.locator('[data-testid="medication-card"]'))
    .toHaveCount(3, { timeout: 5000 });
});
```

```
test("Step 3: Interaction alert visible after Ibuprofen upload", async ({ page
}) => {
  await page.goto("/dashboard/medications");
  const uploadZone = page.locator('[data-testid="upload-zone"]');
  await
uploadZone.setInputFiles("tests/fixtures/ibuprofen_prescription.jpg");
```

```

// Interaction alert MUST appear — no conditional waiting
await expect(page.locator('[data-testid="interaction-alert"]'))
  .toBeVisible({ timeout: 15000 });
await expect(page.locator('[data-testid="interaction-severity"]'))
  .toContainText("HIGH");
});

test("Step 7: Generate health summary PDF", async ({ page }) => {
  const downloadPromise = page.waitForEvent("download");
  await page.click('[data-testid="generate-summary-btn"]');
  const download = await downloadPromise;

  expect(download.suggestedFilename()).toMatch(/CareOrbit_Summary_/);
});
});
```

```

```

_____
_____
_____
_____
CI/CD PIPELINE — .github/workflows/ci.yml
_____
_____
_____
_____

```

Generate a GitHub Actions workflow with these EXACT stages in order:

Stage 1 — SAFETY (runs first, blocks all subsequent stages if fails):

- Run tests/false\_positive\_negative/ with -m "critical"
- Run tests/security/test\_cross\_patient\_rbac.py
- Run tests/security/test\_audit\_append\_only.py
- If ANY fail → BLOCK pipeline, alert Slack #clinical-safety channel

Stage 2 — UNIT (runs if Stage 1 passes):

- tests/unit/ with coverage >= 90%
- Fail fast: -x flag



Stage 3 — INTEGRATION (runs if Stage 2 passes):

- Spin up Postgres 16 service container with pgcrypto
- Run tests/integration/ with real DB
- Run tests/security/ (refresh token, audit trail, RBAC)
- Coverage gate  $\geq 80\%$  for integration code paths

Stage 4 — FUNCTIONAL (runs if Stage 3 passes):

- Run tests/functional/ with mock Azure services
- Run tests/business\_logic/

Stage 5 — E2E (runs on main branch only):

- Build frontend (next build)
- Start backend + frontend
- Run Playwright tests with --reporter=html
- Upload Playwright report as artifact

Stage 6 — DEPLOY (runs if all stages pass, main branch only):

- Deploy backend to Azure Container Apps
- Deploy frontend to Azure Static Web Apps
- Run smoke tests against production

Generate the complete ci.yml file.

```
#####
#####
#####
```

## FINAL DELIVERABLES CHECKLIST

```
#####
#####
#####
```

Before declaring implementation complete, verify:

- ☐ pytest runs with zero warnings (not just zero failures)
- ☐ pytest --co (collect-only) shows  $\geq 875$  test items
- ☐ coverage report shows  $\geq 80\%$  overall,  $\geq 90\%$  for security modules

- No test contains: "if response.status\_code", "if mock.called", "pass" as body
- All Azure mock paths verified against real SDK import paths
- Reminder tests: create → hard assert 200/201 → delete by real ID
- Encryption tests: assert call\_count >= 2 (not if .called)
- Subscription tests: use standard TestClient (no include\_router hack)
- Interaction tests: wiring vs algorithm split applied
- P0 count = 17 (not 14 — v0.3.0 promotions applied)
- caregiver limit test = @xfail(strict=True) with gate mock, not pass body
- patient\_confirmed in PHASE1\_SOURCES (not Phase 2)
- 0.05 floor test ABSENT (was hallucinated, deleted)
- Playwright E2E: Ramesh journey completes with real interaction alert
- CI pipeline: Safety stage runs FIRST and blocks on failure
- README includes: local dev setup, how to run each test stage,  
Azure resource provisioning guide, environment variables reference